Residual Methods Under Model Mismatch

2026-04-01

Method Goal

The residual notebook family adds a second control channel on top of baseline MPC. The MPC still computes a feasible baseline move, but a TD3 policy proposes an extra residual move that is clipped by mismatch-aware safety bounds before it is applied. The intent is to preserve the optimization structure of MPC while letting RL compensate for mismatch that the linear model does not capture well.

Notebook Sources Covered

- `RL_assisted_MPC_residual_model_mismatch.ipynb`
- `RL_assisted_MPC_residual_model_mismatch1.ipynb`
- `RL_assisted_MPC_residual_model_mismatch2.ipynb`
- `RL_assisted_MPC_residual_model_mismatch_multi.ipynb`

State, Action, Reward, And Update Logic

The residual family augments the standard scaled state with innovation and tracking error terms:

$$s_t = \begin{bmatrix} \tilde{x}_t \\ \tilde{y}_{\mathrm{sp},t} \\ \tilde{u}_t^{\mathrm{dev}} \\ \mathrm{innov}_t \\ e_t^{\mathrm{track}} \end{bmatrix},$$

where

$$\mathrm{innov}_t = y_t^{\mathrm{scaled}} - C_{\mathrm{aug}}\hat{x}_t, \qquad e_t^{\mathrm{track}} = y_t^{\mathrm{scaled}} - y_{\mathrm{sp}}^{(t)}.$$

The agent proposes a residual move $\Delta u_t^{\mathrm{res}}$ in scaled space. That proposal is not applied directly. It is clipped by two intersected bounds:

- a magnitude bound relative to the MPC move,

- a safety bound given by actuator headroom around the baseline input.

The applied control is therefore

$$u_t^{\text{applied}} = u_t^{\text{base}} + \Delta u_t^{\text{res,exec}},$$

where $\Delta u_t^{\text{res,exec}}$ is the clipped residual that actually survives the bound intersection.

Mismatch Bands And Residual Gating

The notebook family defines mismatch bands in scaled output coordinates:

$$\text{band}_{\text{scaled}} = \frac{\max(k_{\text{rel}}|y_{\text{sp}}^{\text{phys}}|, \text{band_floor}^{\text{phys}})}{\text{data_max} - \text{data_min}}.$$

It also defines soft and hard gating helpers that estimate how far the current tracking error is outside the allowed band.

```
def compute_band_scaled(y_sp_phys, data_min, data_max, n_inputs,
                        k_rel, band_floor_phys):
    dy = np.maximum(data_max[n_inputs:] - data_min[n_inputs:], 1e-12)
    band_phys = np.maximum(k_rel * np.abs(y_sp_phys), band_floor_phys)
    return band_phys / dy

def residual_outside_weight(e_track_scaled, band_scaled,
                            tau_frac=0.7, kind="geom"):
    abs_e = np.abs(e_track_scaled)
    tau = np.maximum(tau_frac * band_scaled, 1e-12)
    x = (band_scaled - abs_e) / tau
    s = 1.0 / (1.0 + np.exp(-x))
    ...
    return 1.0 - w_in
```

Core Residual Control Loop

1. Compute the baseline MPC move from the current observer state.
2. Compute innovation, tracking error, and a mismatch-band measure.
3. Ask the TD3 policy for a raw residual move proposal.
4. Build a magnitude envelope around the baseline MPC move.
5. Intersect that envelope with actuator headroom bounds.
6. Clip the residual move, apply the corrected control, and log the executed residual rather than the raw proposal.
7. Store the executed residual action in replay so the TD3 buffer reflects what the plant actually saw.

Key Code Paths

Notebook-local residual safety logic

```
u_base = sol.x[:n_inputs] + ss_scaled_inputs
u_base = np.clip(u_base, u_min_scaled_abs, u_max_scaled_abs)

delta_u_mpc = (u_base - u_current).astype(np.float32)
delta_u_res_raw = map_to_bounds(a_raw, low_res, high_res).astype(np.float32)

mag = (rho * beta_res) * (np.abs(delta_u_mpc) + du0_res)
low_mag = -mag
high_mag = mag

low_safe = (u_min_scaled_abs - u_base).astype(np.float32)
high_safe = (u_max_scaled_abs - u_base).astype(np.float32)

low_bound = np.maximum(low_mag, low_safe)
high_bound = np.minimum(high_mag, high_safe)
delta_u_res = np.clip(delta_u_res_raw, low_bound, high_bound)
```

Notebook-local replay action logic

```
u_applied_scaled_abs = u_base + delta_u_res
u_applied_scaled_abs = np.clip(u_applied_scaled_abs,
                               u_min_scaled_abs, u_max_scaled_abs)

delta_u_res_exec = (u_applied_scaled_abs - u_base).astype(np.float32)
a_exec = map_from_bounds(delta_u_res_exec, low_res, high_res).astype(np.float32)
a_exec = np.clip(a_exec, -1.0, 1.0).astype(np.float32)
```

Variants Covered

- The base residual mismatch notebook defines the main residual scaffold.
- Variants `1` and `2` keep the same pattern while changing disturbance or mismatch schedules.
- The `multi` variant extends the idea to a more involved residual setting while preserving the same core ingredients: innovation terms, mismatch bands, clipped residual execution, and replay of executed moves.

Limitations, Caveats, And Open Questions

- This family depends on several notebook globals such as `TEST_CYCLE`, `IC_opt`, `bnds`, `cons`, `k_rel`, and `band_floor_phys`.
- Residual replay stores the executed clipped action, not the raw action. That is intentional and must be preserved in refactors.

- Additional mismatch scheduling includes changes to `CMf`, not just the shared Q_i, Q_s, and hA schedules.
- This method is explicitly notebook-local and should not be inferred from stale helper modules.